TUPLES, DOCSTRINGS & TESTS

INFO1 Mission 5 (S6)

Jean-Luc Bibaud



Tuples?

```
TUPLE_NAME = (value, variable, ...)
```

A tuple is a **collection of data**, like a list.

So why not use lists?

- Tuples, strings and ints **cannot change** (immutable): no accidental changes
- So they take **less memory** (and are a bit faster)

```
point = (3, 4)
point[0]
→ 3
point[0] = 5
→ TypeError: 'tuple' object does not
  support item assignment
type((5))
→ <class 'int'>
type((5,))    # comma!
→ <class 'tuple'>
```

Tuple assignment

How it works

The variable names on the left are **unpacked** from the values on the right, in order. Same number on both sides.

Packing and unpacking

```
b = ("Bob", 19, "CS")       # packing
(name, age, studies) = b   # unpacking

print(name)
→ Bob
print(age)
→ 19
```

Handy tricks

```
x, y = 1, 2    # brackets optional
x, y = y, x    # swap!
print(x, y)
→ 2 1

name, age = b
→ ValueError: too many values to
  unpack (expected 2)
```

How to properly document a function

The template

```
def function_name(arguments):
    """One short line describing the function.

    Longer text to describe the function.

    Args:
        arg_1 (type): short description.
        ... (every argument is described) ...
        arg_n (type): short description.

    Returns:
        type: what the function returns.
    """
    function code
```

A real example

```
def area(w, h):
    """Return the area of a rectangle.

    Works for any unit, as long as both
    sides use the same one.

    Args:
        w (float): width of the rectangle.
        h (float): height of the rectangle.

    Returns:
        float: the area, w * h.
    """
    return w * h
```

Tip: `help(area)` prints this text, and Thonny shows it when you hover the function.

How to code tests in Python

```
assert CONDITION, "message if the condition is not true"
```

```
def area(w, h):
    return w * h

assert area(2, 3) == 6, "2 x 3 should be 6"
→ nothing happens: the test passes
assert area(0, 5) == 0, "a zero side gives 0"
→ passes too
assert area(2, 3) == 5, "2 x 3 should be 6"
→ AssertionError: 2 x 3 should be 6
```

Good habits

1. **No output** means every test passed.
2. Test normal cases **and** edge cases: 0, negative numbers, empty strings…
3. **assert is not a function:** `assert(x == 1, "msg")` is always true!

Who is executing?

```python
# m.py
def greet():
    print("greet")

print("hi")
if __name__ == "__main__":
    greet()
```

What is `__name__`?

For the file you run, __name__ is `"__main__"`.

When a file is imported, __name__ is its module name, so the code under the if **does not run** (import m, from m import *, any import).

Run it directly

```
$ python m.py
→ hi
greet
```

Import it from another file

```
import m
→ hi
  (greet() is not called)
```